CENTRO UNIVERSITÁRIO DE MARINGÁ – UNICESUMAR
ENGENHARIA DE SOFTWARE | ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ATIVIDADE DE ESTUDO PROGRAMADA (AEP) – 4º SEMESTRE (2026.2)

PROJETO MOVECITY:
APLICAÇÃO WEB PARA MOBILIDADE URBANA SUSTENTÁVEL

Documento de Entrega da 2ª Etapa – Implementação, Rigor e Integração

Equipe de Desenvolvimento	RA
Integrante 1: Miguel Souza Colis	25014158-2
Integrante 2: Arthur Riki Farias Tanaka	25224963-2
Integrante 3: João Gustavo Zanoni Gonçalves	25357434-2

ODS vinculados: ODS 11 – Cidades e Comunidades Sustentáveis (principal) e ODS 13 – Ação contra a Mudança Global do Clima (complementar)

Repositório GitHub (público) com o código-fonte completo:

https://github.com/Arthurtanaka/AEP-ESOFT4S

Maringá, novembro de 2026

1. APRESENTAÇÃO

Este documento acompanha a entrega da 2ª etapa da AEP, cujo foco é o software funcional e integrado. O código-fonte completo, o script SQL de criação do banco, os diagramas e as instruções de execução estão no repositório público do projeto, cujo link consta na capa. O documento resume o que foi implementado, registra as alterações e melhorias feitas em relação à 1ª entrega, relaciona cada critério da atividade à evidência correspondente no repositório e explica como executar o sistema.

A 1ª entrega foi mantida como referência de planejamento: o escopo, as tecnologias (Java e MySQL), a arquitetura em camadas e o alinhamento ao ODS 11 permanecem os mesmos. As alterações descritas na Seção 3 são refinamentos que tornaram o planejamento mais coerente e implementável, e a versão revisada do documento da 1ª entrega está na pasta /docs do repositório.

2. O QUE FOI ENTREGUE

- **Aplicação web funcional:** back-end em Java (49 arquivos-fonte, sem frameworks, usando o servidor HTTP que já acompanha o Java) e interface em HTML5, CSS3 e JavaScript, que conversam por HTTP com troca de dados em JSON.
- **Arquitetura em camadas:** pacotes model, dao, service, controller e util, conforme o planejado na 1ª entrega.
- **Banco de dados MySQL** com conexão ativa por JDBC. O script de criação das tabelas (database/schema.sql) é compatível com as classes Java, e um script opcional (database/dados_exemplo.sql) carrega dados de demonstração.
- **Os dez requisitos do sistema (RQ01 a RQ10)** implementados: cadastro e login, CRUD de rotas e de meios de transporte, consulta de alternativas, comparação por tempo, distância e impacto ambiental, destaque da alternativa mais sustentável, favoritos, histórico e alertas.
- **Nove regras de negócio (RN01 a RN09)** aplicadas na camada Service, incluindo permissões por perfil, senha armazenada apenas como hash e transações na gravação de rotas e trechos.
- **Instruções de execução** para Windows, Linux e macOS (README.md e scripts executar.bat e executar.sh).

3. ALTERAÇÕES E MELHORIAS EM RELAÇÃO À 1ª ENTREGA

A tabela compara o que constava na 1ª entrega com o que foi implementado e explica o motivo de cada ajuste.

Item	1ª entrega	2ª entrega (implementado)	Motivo
Lista de requisitos	RF01 a RF08 (8 requisitos)	RQ01 a RQ10 (10 requisitos). Os 8 originais foram mantidos: RF01 = RQ01; RF02 e RF03 = RQ05; RF04 = RQ06; RF05 = RQ07; RF06 = RQ08; RF07 = RQ09; RF08 = RQ10. Foram acrescentados RQ02 (login), RQ03 (CRUD de rotas) e RQ04 (CRUD de transportes).	O CRUD completo da entidade principal e a gestão do catálogo exigem os requisitos novos. A palavra “funcionais” foi retirada da nomenclatura, conforme a orientação da atividade.
Entidade principal	Não declarada de forma explícita	**Rota**, com CRUD completo (banco, regras, API e tela).	Deixar claro o alvo do CRUD exigido na 2ª entrega.
Diagrama de classes	Rota com distanciaKm armazenada; Transporte com atributos genéricos	Distância, tempo e impacto da Rota são calculados pela soma dos trechos; Transporte passou a ter velocidade média e emissão de CO2; Carro tem fator de congestionamento e Onibus tem capacidade. Herança, polimorfismo e composição 1:N foram mantidos.	Evitar dado duplicado e viabilizar os cálculos de tempo e impacto (RQ06) de forma polimórfica.
DER	Tabela rota ligada ao usuário; alerta ligado ao usuário	Rota independente do usuário; usuário se liga à rota por favorito e histórico; alerta ligado à rota e ao administrador que o registrou; tabela transporte com velocidade, emissão, capacidade e fator.	Coerência entre o diagrama de classes, o DER e o texto do documento.
Arquitetura	Model, DAO e Service, com front-end e API REST na figura	Camada Controller explícita (um controller por recurso, todos herdando de BaseController) e interface web servida pelo próprio sistema.	Materializar a ligação entre a interface e as regras de negócio prevista na figura da arquitetura.
Regras de negócio	Citadas de forma geral	RN01 a RN09 definidas e implementadas (e-mail único, hash de senha, permissões, rota com trecho, soma dos trechos, desempate da opção sustentável, histórico apenas de autenticados, favorito único e alerta inativo oculto).	Tornar o comportamento verificável e testável.
Alinhamento aos ODS	ODS 11 de forma geral	ODS 11 com as metas 11.2 e 11.6 relacionadas aos requisitos, e ODS 13 como complemento.	Responder ao retorno de que o alinhamento estava superficial.
Cronograma	Somente a 1ª etapa	Fases 1 e 2, com datas e responsáveis, espelhado no README.	Cobrir o planejamento das duas entregas.

Item	1ª entrega	2ª entrega (implementado)	Motivo
Repositório	Estrutura com README, docs, database e src	Mesma estrutura, agora com código, scripts de execução, dados de exemplo, mapa de avaliação no README e roteiro e script de testes em /docs.	Facilitar a auditoria e a execução em outros ambientes.

Tabela 1: Alterações e melhorias em relação à 1ª entrega.

4. ATENDIMENTO AOS CRITÉRIOS DA 2ª ENTREGA

Critério (peso)	Evidência no repositório
Aderência ao planejamento (0,2)	Todos os requisitos RQ01 a RQ10 estão implementados e mapeados para classe, API e tela na seção “Requisitos e onde estão no código” do README. Eles cobrem os RF01 a RF08 da 1ª entrega (Tabela 1). Linguagem, banco e arquitetura são os planejados.
Pilares de POO (0,3)	Classes abstratas Usuario, Transporte e BaseController; interface CrudDAO<T>; herança em Passageiro e Administrador, Carro, Onibus e Bicicleta e nos oito controllers; polimorfismo com @Override (autenticar, getPerfil, calcularTempo, calcularImpacto, tratar e os métodos de CrudDAO); composição Rota–Trecho. Código tipado, com nomes em CamelCase.
CRUD funcional (0,3)	Entidade Rota: RotaDAO, RotaService, RotaController e tela Administração > Rotas, com Create, Read, Update e Delete. Cada operação é verificada pelo script docs/testes/teste_api.py.
Persistência em BD (0,2)	Conexão JDBC em dao/ConexaoBanco.java; script database/schema.sql na pasta /database, compatível com as classes (tabela de correspondência no README); gravações com transação; os dados permanecem após reiniciar o sistema.
Ambiente (Sistemas Operacionais)	README com pré-requisitos, passo a passo, scripts para Windows, Linux e macOS, configuração por arquivo local fora do repositório e solução dos problemas mais comuns.
Contribuição de todos	Histórico de commits do repositório; divisão de responsabilidades descrita na Seção 7.

Tabela 2: Critérios da atividade e evidências.

5. COMO ACESSAR E EXECUTAR

O passo a passo completo está no README.md do repositório. Em resumo:

- Instalar o JDK 17 ou superior e o MySQL 8.0 ou superior.
- Baixar o repositório (git clone ou Download ZIP).
- Criar o banco com **database/schema.sql** e, opcionalmente, carregar os dados de demonstração com **database/dados_exemplo.sql**.

- Colocar o driver MySQL Connector/J (arquivo .jar) na pasta **lib/** e, se o MySQL tiver senha, criar o **config.local.properties** a partir do config.example.properties.
- Executar **executar.bat** (Windows) ou **./executar.sh** (Linux e macOS) e abrir **http://localhost:8080** no navegador.

Usuários de demonstração (criados pelo dados_exemplo.sql): administrador **admin@movecity.com** (senha admin123) e passageiro **passageiro@movecity.com** (senha senha123). Para cadastrar um novo administrador é necessário o código de administrador configurado no sistema (padrão movecity2026). Consulta sugerida para a demonstração: origem **Centro**, destino **Zona Norte**.

6. TESTES REALIZADOS

- **Teste automatizado da API** (docs/testes/teste_api.py): 66 verificações que cobrem RQ01 a RQ10, RN01 a RN09, permissões por perfil, validações, o CRUD completo de Rota e de Transporte e o tratamento de erros. Resultado: todas aprovadas.
- **Teste da interface** em navegador simulado: 30 verificações (consulta, login, favoritos, histórico e o CRUD de rotas, transportes e alertas na tela de administração), todas aprovadas.
- **Conferência manual dos cálculos:** por exemplo, a rota de carro de 12 km resulta em 31,2 minutos e 1.440 g de CO2, e a rota de ônibus de 9 km combinada com bicicleta de 3 km resulta em 44 minutos e 630 g de CO2, destacada como a mais sustentável.
- **Roteiro manual** de 24 passos em docs/ROTEIRO_DE_TESTES.md, para conferência pela interface.

Os testes foram executados em Linux (Ubuntu 24.04), com JDK 21 e MySQL Server 8.0. O código compila também com JDK 17.

7. DIVISÃO DE RESPONSABILIDADES

A divisão segue o cronograma do projeto, e a contribuição de cada integrante está registrada no histórico de commits do repositório.

Integrante	Responsabilidades
Miguel Souza Colis	Pacote model (classes de domínio) e interface web (HTML, CSS e JavaScript).
Arthur Riki Farias Tanaka	Repositório, banco de dados (schema.sql e dados de exemplo), pacote dao, pacote util, configuração e scripts de execução.
João Gustavo Zanoni Gonçalves	Pacote service (regras de negócio), CRUD de Rota e pacote controller (API).

Tabela 3: Responsabilidades da equipe.

8. CONCLUSÃO

A 2ª etapa transformou o planejamento da 1ª entrega em um sistema funcional: o MoveCity permite consultar, comparar e escolher rotas de deslocamento considerando tempo, distância e impacto ambiental, e destaca a alternativa mais sustentável, em alinhamento às metas 11.2 e 11.6 do ODS 11. O sistema aplica os pilares da Programação Orientada a Objetos, realiza o CRUD completo da entidade principal sobre um banco de dados MySQL e pode ser executado nos principais sistemas operacionais com as instruções do repositório.

Como evolução futura, a equipe pretende integrar uma API de mapas para calcular distâncias reais, incluir novos meios de transporte, como caminhada e metrô, e acrescentar testes automatizados em Java.